Culpa

Who pays when an AI agent buys the wrong thing?

Culpa works out who is at fault: the buyer, the merchant or the company behind the agent. Then it settles the dispute on PayPal, with the evidence to back it up.

98.7%
of verdicts correct on 150 synthetic disputes

0%
of money sent to the wrong party in that benchmark

126 / 126
scenarios built from real PayPal and card rules

Product flow for judges. PayPal AI Hackathon 2026.

The problem, and the flow

Most agent safety tools stop a bad payment before it happens. Nothing decides fault after the money has moved.

When an agent's purchase goes wrong, today's dispute system has no way to tell why. The instruction may have been unclear, the agent may have ignored it, the merchant may have shipped something different, or the buyer may be blaming the agent to get a refund. Culpa records the evidence at purchase so that question can be answered later.

From instruction to settlement

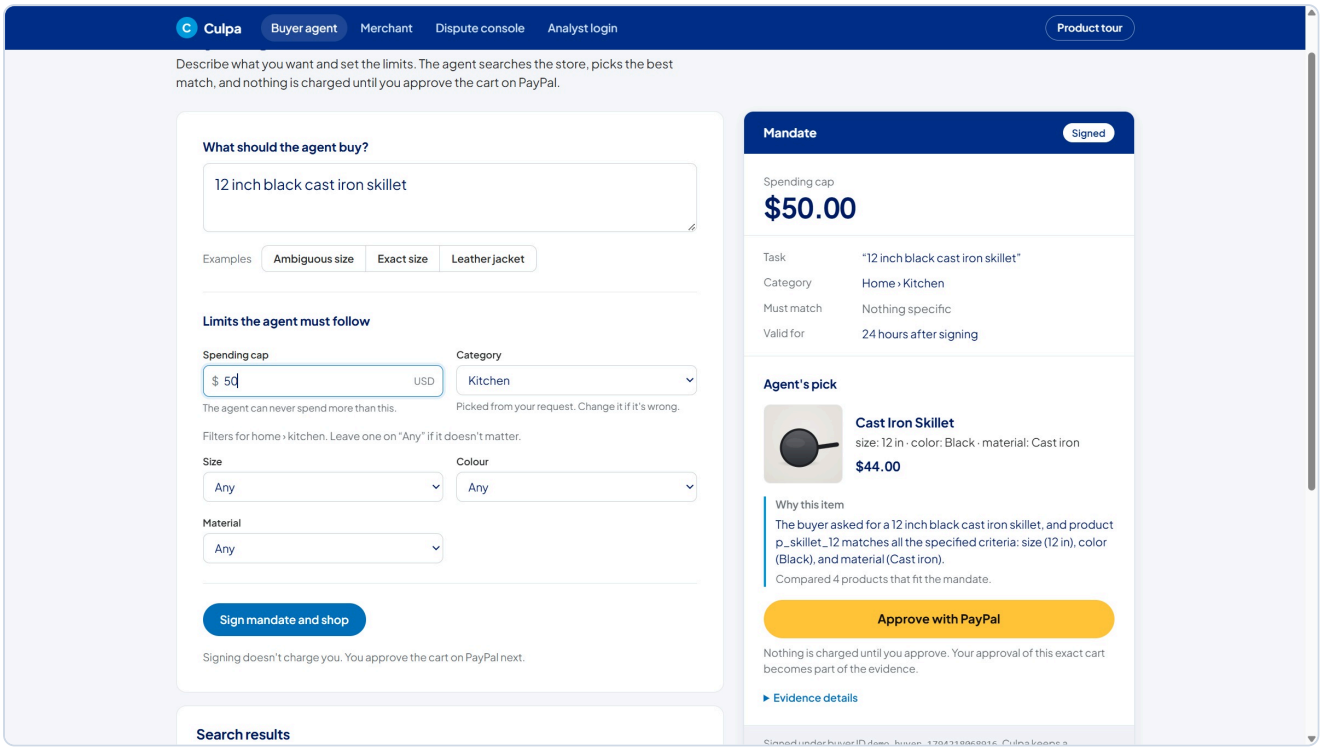
1. **The buyer signs a mandate.** What to buy, a spending cap, a category and any attributes that must match. It is signed so it cannot change later.
2. **The agent searches the store.** Google Gemini ranks products inside the mandate and picks one. The buyer can keep the pick or choose another.
3. **The buyer pays on PayPal.** The agent creates the order with the PayPal Agent Toolkit and the buyer approves the exact cart.
4. **Culpa fingerprints the evidence.** The mandate, the agent's decision and the listing at purchase are hashed into one tamper-evident root.
5. **A dispute arrives and four checks run.** Rules against the mandate, the listing against delivery, a replay of the agent's decision, and the buyer's claim against the record.
6. **Culpa gives a verdict.** Fault follows the cause, not a vote. Other signals change only the confidence. Every verdict names a plain reason and the real-world rule it follows.
7. **Culpa settles or holds it.** A clear verdict at 80% confidence or higher runs on PayPal with no analyst. Anything else waits for a person.

Who is at fault

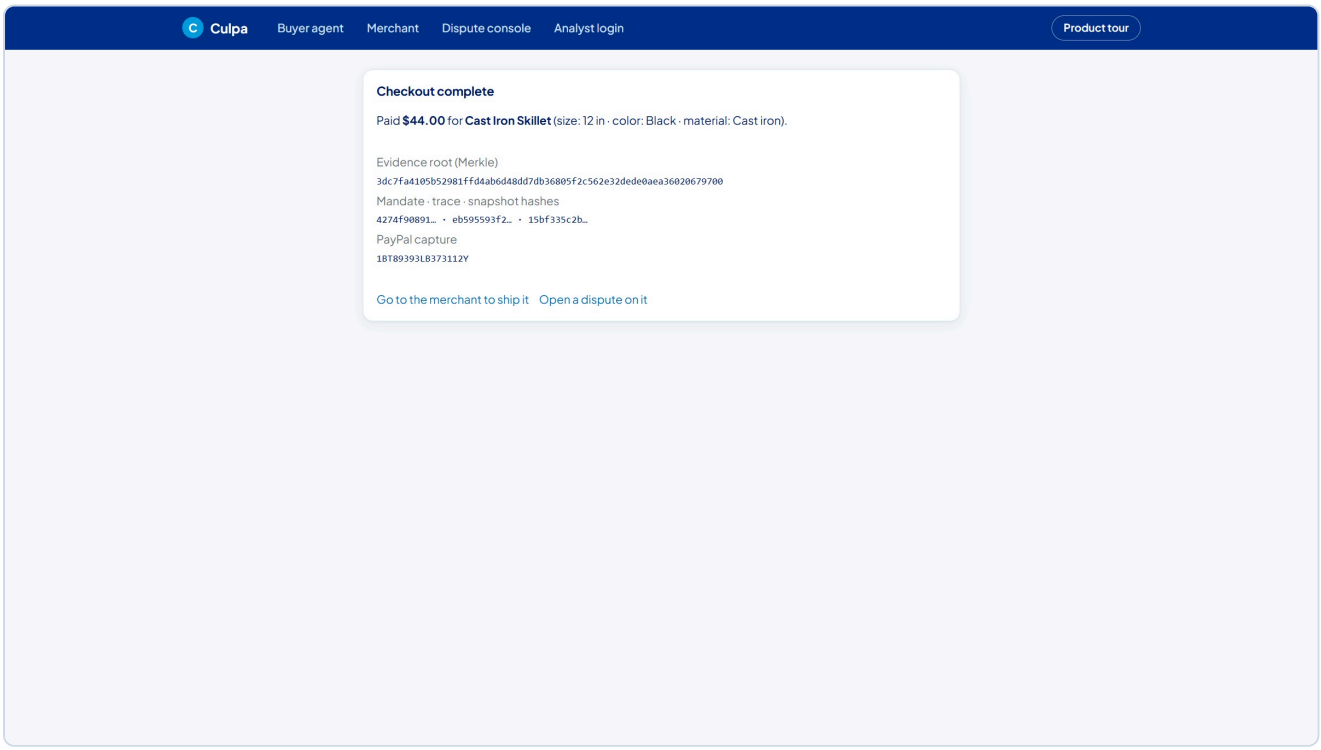
Cause found	Who bears it
Agent broke the mandate, or chose the wrong item	Agent provider
Item not delivered, or differs from the listing	Merchant
Instruction was genuinely ambiguous	Buyer and agent provider share it
Claim contradicts the record (friendly fraud)	Buyer

The purchase

A $44.00 cast iron skillet, bought in the PayPal sandbox.



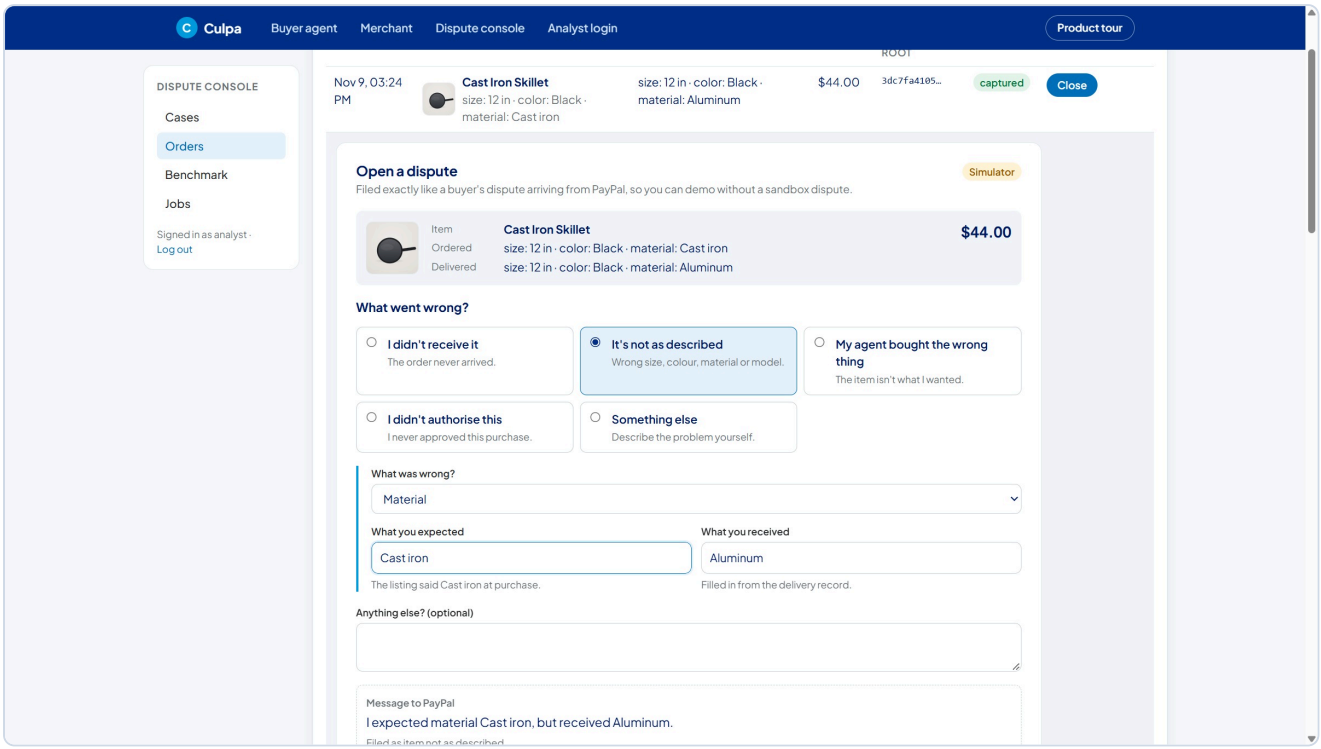
The buyer sets the request and a $50.00 cap. Signing locks the mandate, and the agent shows its pick with its reasoning.



After approval on PayPal, the payment is captured and the evidence root is stored with the order.

The dispute form

Open it from Dispute console, then Orders, then Open dispute on a captured order.



The form shows the item as ordered and as delivered. Here the buyer chose "It's not as described", picked Material and entered what they expected. "What you received" is filled in from the delivery record.

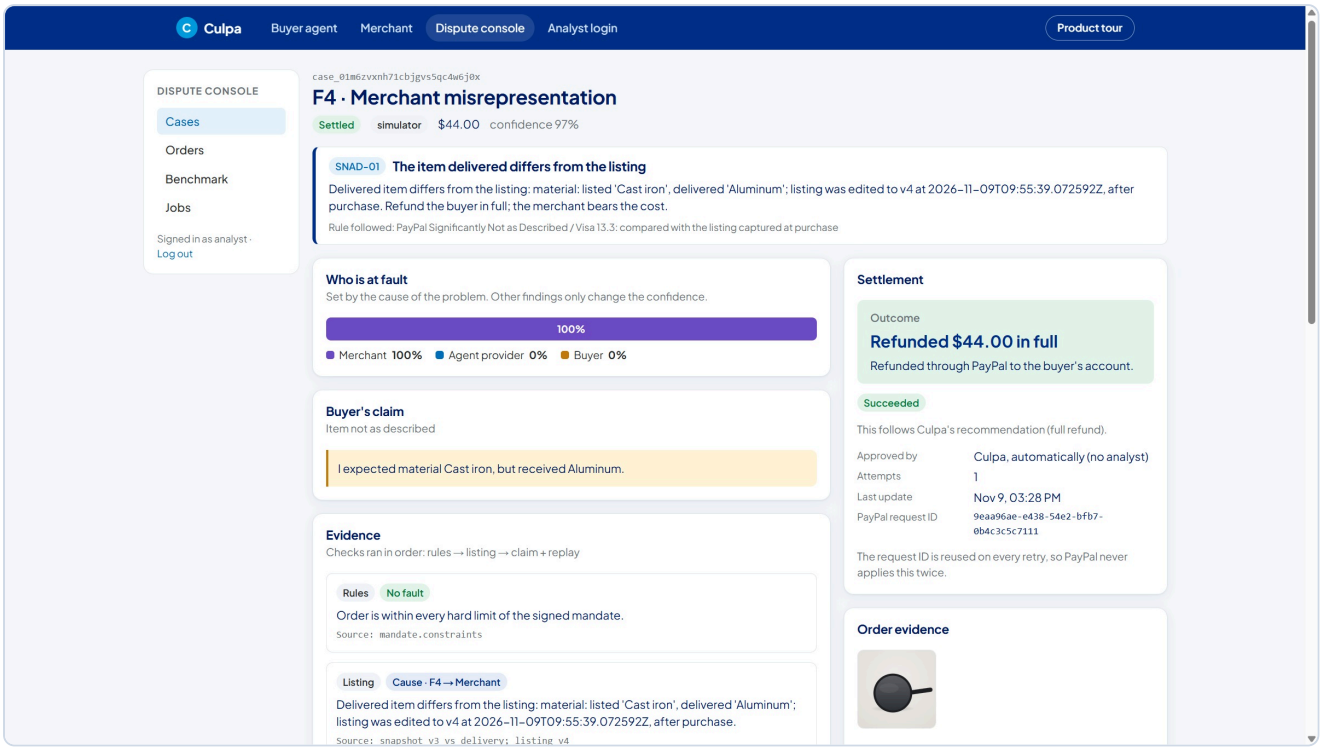
What went wrong? The five options

Option	What the buyer adds	Filed under	What Culpa compares
I didn't receive it	Nothing required	Item not received	The claim against the delivery record
It's not as described	Wrong detail, what they expected, what they received	Item not as described	What was delivered against the listing at purchase
My agent bought the wrong thing	Optional: what they wanted instead	Item not as described	The mandate, the cart they approved, and a replay of the agent
I didn't authorise this	Nothing required	Unauthorised	The claim against the cart the buyer approved
Something else	Their own description, required	Other	The written claim. If the records cannot prove it, a person decides

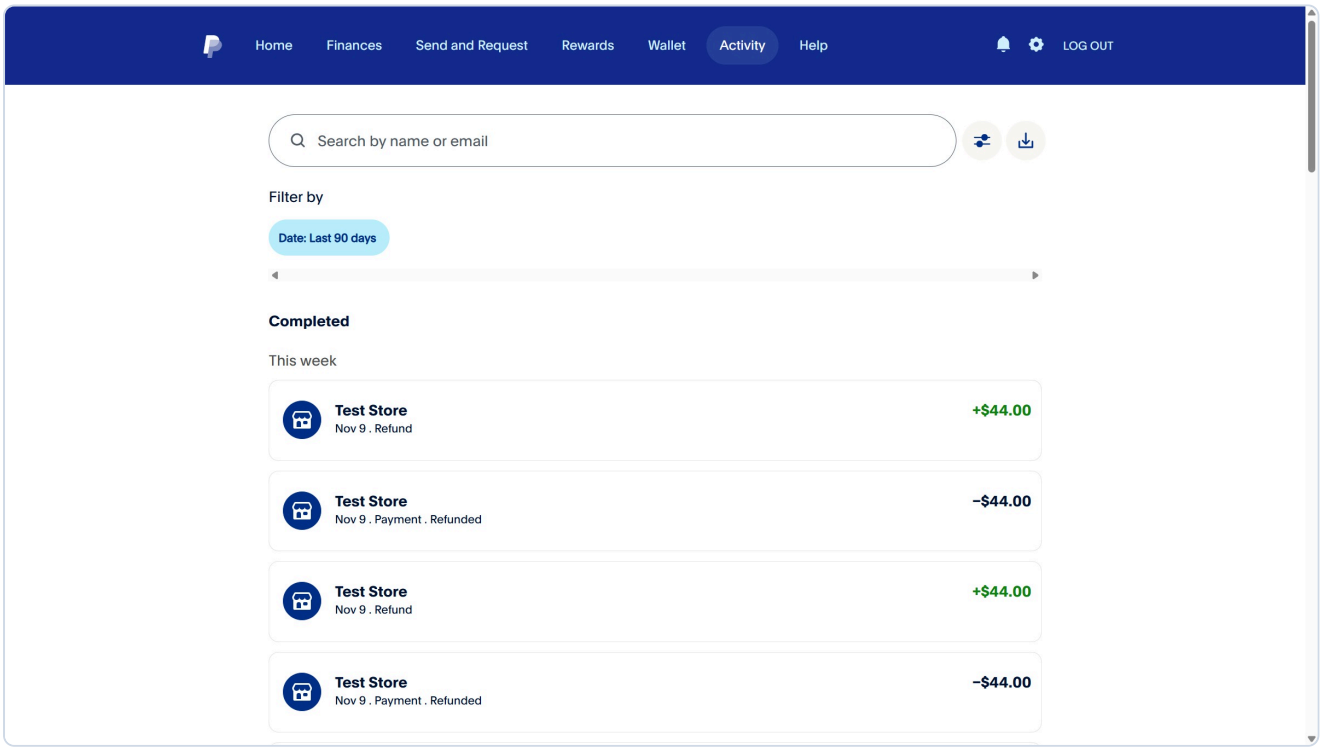
Every option also has a free-text box. Before filing, the buyer sees the exact message and the PayPal reason it will go under. The form is labelled Simulator: it files a dispute the same way one arriving from PayPal is handled, so the flow can be shown without a sandbox dispute.

Story 1: a payment refund, with no human

The merchant changed the listing after the sale, and the buyer received something different.



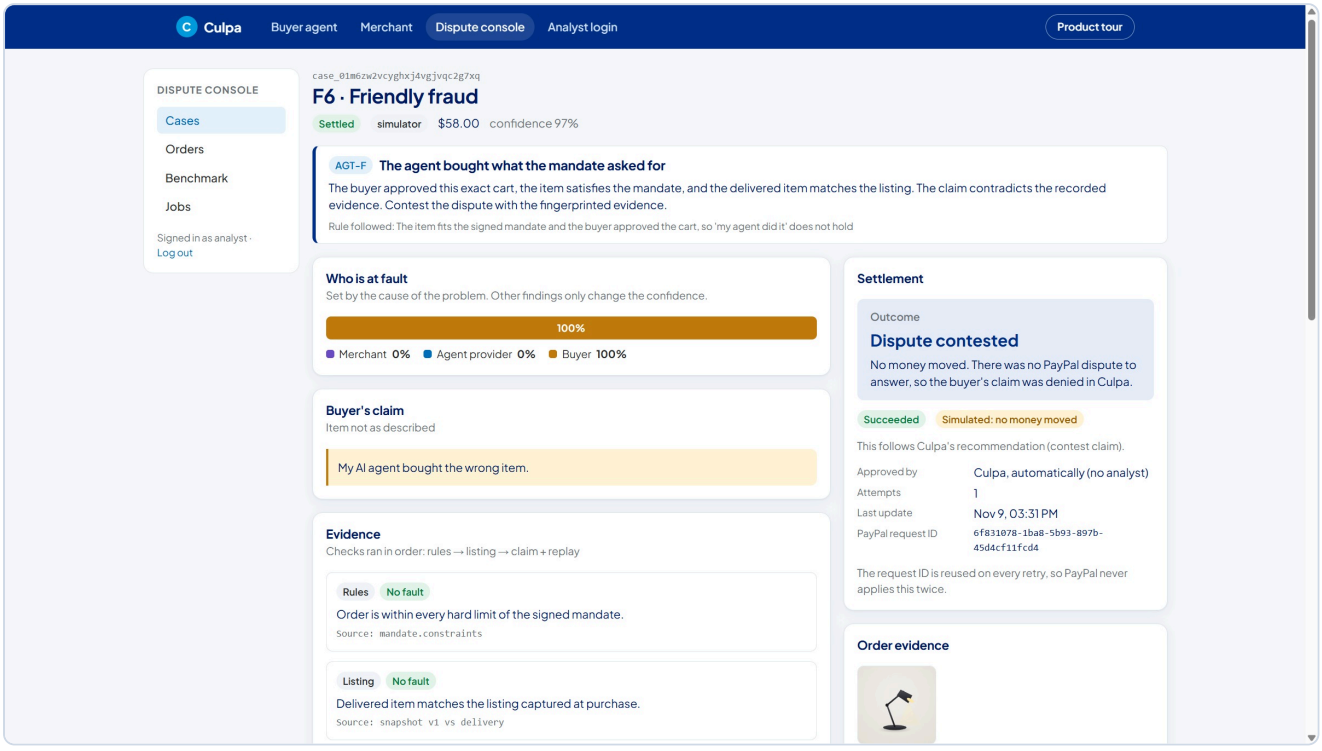
Merchant 100% at fault, 97% confidence. The refund was approved by "Culpa, automatically (no analyst)". This dispute was filed through Culpa's simulator form, which is why the case says "simulator". The refund itself is a real sandbox refund.



The buyer's own PayPal sandbox activity: +$44.00 Refund, and the payment marked Refunded. Two $44.00 refunds are listed because the flow was run twice.

Story 2: a dispute Culpa contests

A different order: a $58.00 desk lamp, delivered exactly as listed. The buyer says the agent bought the wrong item.



Buyer 100% at fault, 97% confidence, contested automatically. No money moved. Because the claim came through the simulator, there was no PayPal dispute to answer, and the case says so.

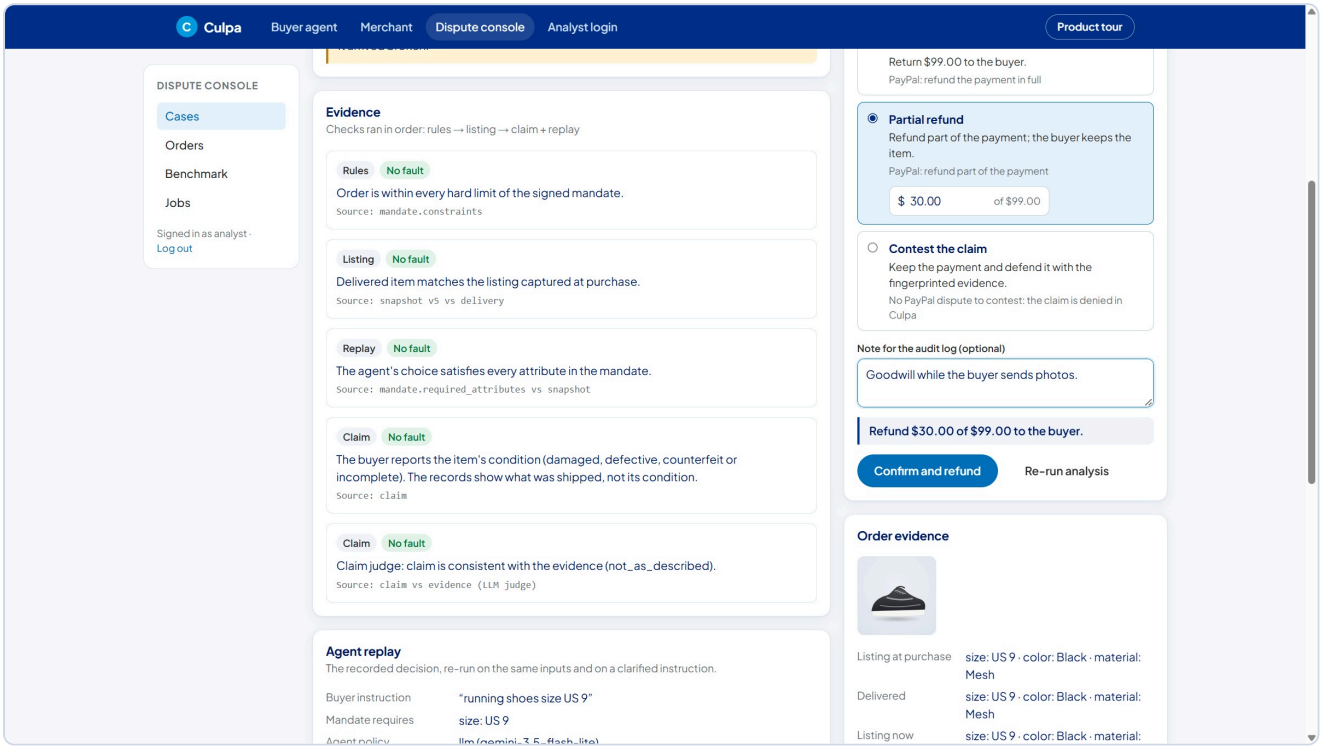
When Culpa acts, and when it waits

Situation	What happens
Clear verdict, 80% confidence or higher	Settled automatically: full refund, partial refund or contest
Confidence below 80%	Held for an analyst
Records cannot prove it: arrived broken, counterfeit, duplicate charge, refund never arrived	No blame assigned. Held for an analyst, with a note on the evidence needed
Order already settled under another case	Held for an analyst
Buyer describes an item the records say never arrived	Held for an analyst
Partial refund on a real PayPal dispute	Sent as an offer. Money moves only if the buyer accepts

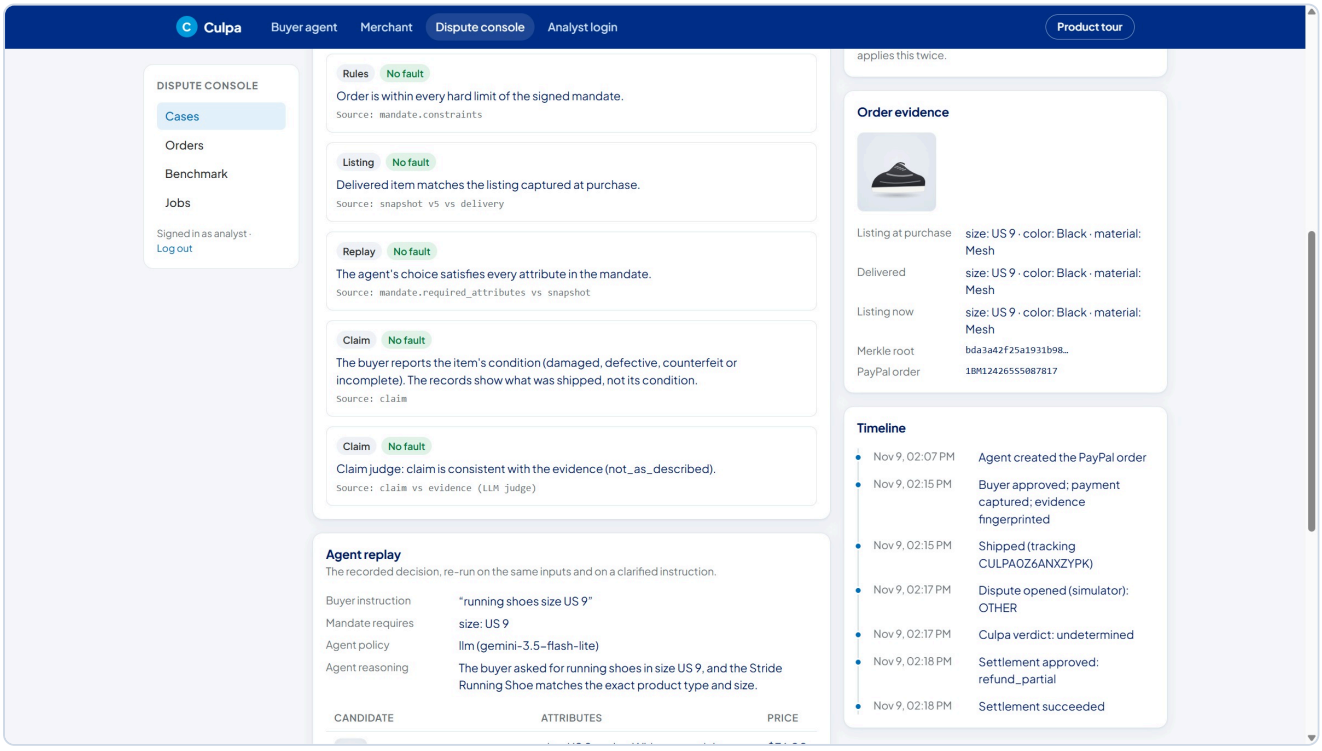
Story 3: a manual refund, decided by a person

A different order: $99.00 running shoes. The buyer chose "Something else" and wrote "It arrived broken."

Records show what shipped, not its condition. All the checks found no fault, so Culpa assigned no blame and held the case for an analyst.



The manual settlement screen. The analyst can refund in full, refund part of the payment or contest the claim. Here they choose a partial refund of $30.00 and add a note for the audit log. The screen states what will happen before they confirm.



The case timeline: dispute opened under OTHER, verdict undetermined, settlement approved as a partial refund, settlement succeeded. This $30.00 was a real sandbox refund.

Why it can be trusted with money

The AI model never moves money.

- **Gemini only advises.** It receives the buyer's text as quoted data, must answer in a fixed format, has no tools, and its output only adjusts confidence.
- **Code decides.** A written settlement policy chooses what happens to money.
- **Never twice.** Every PayPal request carries an idempotency key that stays the same on every retry.
- **Checked first.** Before money moves, Culpa reads the payment back from PayPal and stops if it was already refunded or the dispute has closed.
- **Same path for people and automation.** Autonomous settlements use the same queue, state machine and retries as an analyst's.
- **Tamper-evident history.** Each audit entry contains the hash of the one before it.
- **No card data.** PayPal hosts payment entry.

Benchmark: AgentDispute-150

System	Accuracy	Money to wrong party	Sent to a person
Reads only the buyer's claim	73.3%	26.7%	0.0%
Culpa without agent replay	65.3%	0.0%	34.7%
Culpa, all four checks	**98.7%**	**0.0%**	**2.7%**

150 synthetic disputes, 25 per fault type, 30 of them deliberately hard. The data is synthetic and shares Culpa's fault taxonomy, so these are in-distribution numbers, not real-world accuracy.

A further 126 scenarios are built from PayPal Buyer Protection and Visa and Mastercard dispute rules, and all 126 pass. The 256 automated backend tests pass.

What has been tested on real PayPal

All runs used the PayPal sandbox with play money. Results were read back from PayPal.

Action	Approved by	Amount	Result
Full refund	Culpa, automatically	$59.00	Succeeded
Full refund (run twice)	Culpa, automatically	$44.00	Succeeded
Full refund	Analyst	$96.00	Succeeded
Partial refunds	Analyst	$4, $30, $50	Succeeded
Full refund (two attempts)	Culpa, automatically	$99.00	Refused by PayPal

Known limits

- **$99.00 refunds.** PayPal answered REFUND_NOT_ALLOWED although the payments were intact. No money moved, Culpa recorded the failure, and the cause is still unconfirmed.
- **Simulated disputes.** The dispute form is Culpa's own, so a contest on those cases moves no money and is labelled simulated.
- **Disputes filed inside PayPal.** The code handles them, through webhooks or a scheduled check of PayPal's dispute list, but they have not yet been tested end to end.
- **Webhooks.** Verification needs PAYPAL_WEBHOOK_ID, which the current Render deployment does not set.

Built with

PayPal Agent Toolkit, Orders v2, Payments refunds and Disputes v1. Google Gemini (gemini-3.5-flash-lite and gemini-3.5-flash). Python 3.11 with FastAPI and SQLAlchemy, Postgres, Redis, Next.js, and Render.

Try it

Setup, demo logins and commands are in START_HERE.md in the repository. For an automatic refund: buy an item, change its material on the merchant page, ship it, and dispute it as not as described.